Explainable AI with Shapley values

Shapley values, rooted in cooperative game theory, are a widely used method for explaining machine learning model predictions with desirable mathematical properties. This hands-on tutorial uses the shap Python package to teach how to compute and interpret Shapley-based explanations across progressively complex models. It serves as a living document, welcoming community feedback and contributions to improve over time.

Outline

Explaining a linear regression model

Explaining a generalized additive regression model

Explaining a non-additive boosted tree model

Explaining a linear logistic regression model

Explaining a non-additive boosted tree logistic regression model

Dealing with correlated input features

Explaining a transformers NLP model

Explaining a linear regression model

Before using Shapley values to explain complicated models, it is helpful to understand how they work for simple models. One of the simplest model types is standard linear regression, and so below we train a linear regression model on the California housing dataset. This dataset consists of 20,640 blocks of houses across California in 1990, where our goal is to predict the natural log of the median home price from 8 different features:

MedInc - median income in block group

HouseAge - median house age in block group

AveRooms - average number of rooms per household

AveBedrms - average number of bedrooms per household

Population - block group population

AveOccup - average number of household members

Latitude - block group latitude

Longitude - block group longitude

In [1]:
```
import sklearn
import shap

# a classic housing price dataset
X, y = shap.datasets.california(n_points=1000)

X100 = shap.utils.sample(X, 100)  # 100 instances for use as the background dist

# a simple linear model
model = sklearn.linear_model.LinearRegression()
model.fit(X, y)
```

Out[1]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

In [2]:
```
(X).head()
```

Out[2]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	L
14740	4.1518	22.0	5.663073	1.075472	1551.0	4.180593	32.58	
10101	5.7796	32.0	6.107226	0.927739	1296.0	3.020979	33.92	
20566	4.3487	29.0	5.930712	1.026217	1554.0	2.910112	38.65	
2670	2.4511	37.0	4.992958	1.316901	390.0	2.746479	33.20	
15709	5.0049	25.0	4.319261	1.039578	649.0	1.712401	37.79	

In [3]:
```
import pandas as pd

y = pd.DataFrame(y, columns=['target'])
y.head
```

Out[3]:
```
<bound method NDFrame.head of      target
0     1.369
1     2.413
2     2.007
3     0.725
4     4.600
..      ...
995   1.855
996   1.058
997   3.435
998   2.259
999   2.683

[1000 rows x 1 columns]>
```

Examining the model coefficients

The most common way of understanding a linear model is to examine the coefficients learned for each feature. These coefficients tell us how much the model output changes when we change each of the input features:

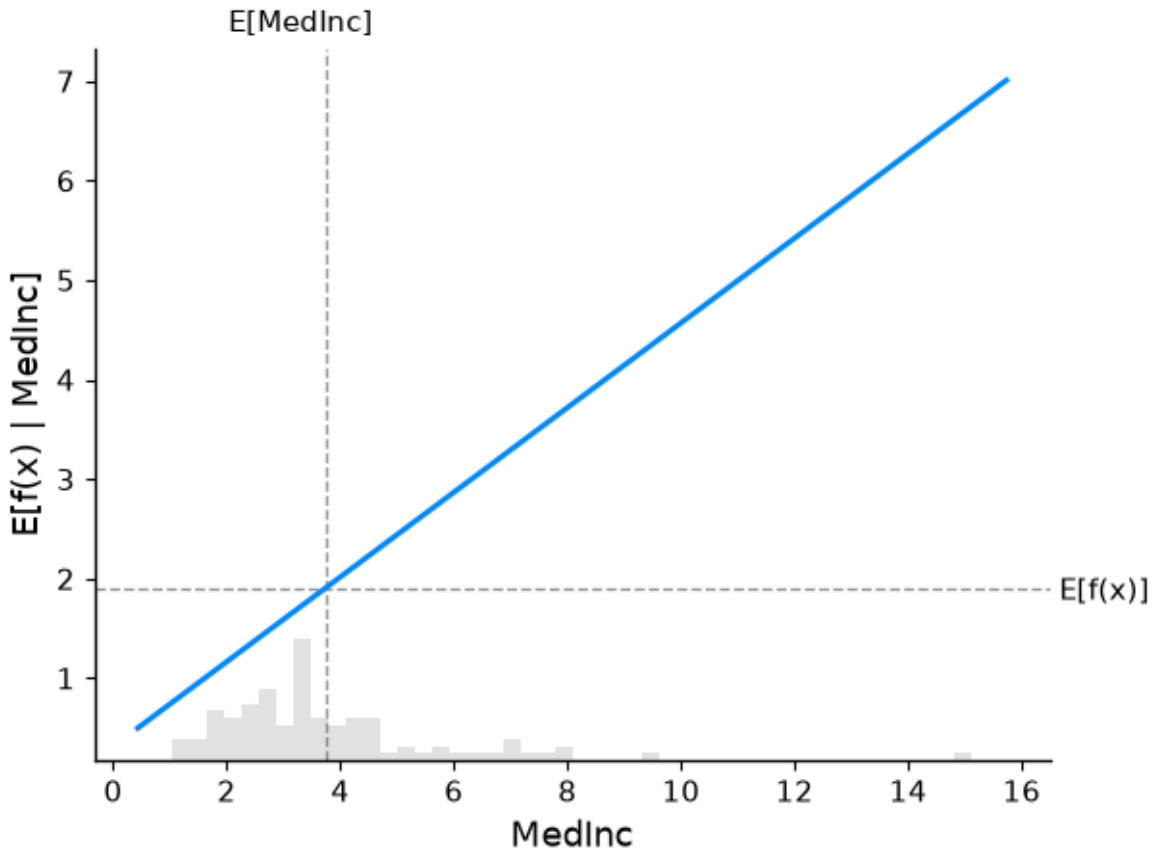
In [4]:
```
print("Model coefficients:\n")
for i in range(X.shape[1]):
    print(X.columns[i], "=", model.coef_[i].round(5))
```

```
Model coefficients:

MedInc = 0.42563
HouseAge = 0.01033
AveRooms = -0.1161
AveBedrms = 0.66385
Population = 3e-05
AveOccup = -0.26096
Latitude = -0.46734
Longitude = -0.46272
```

A more complete picture using partial dependence plots

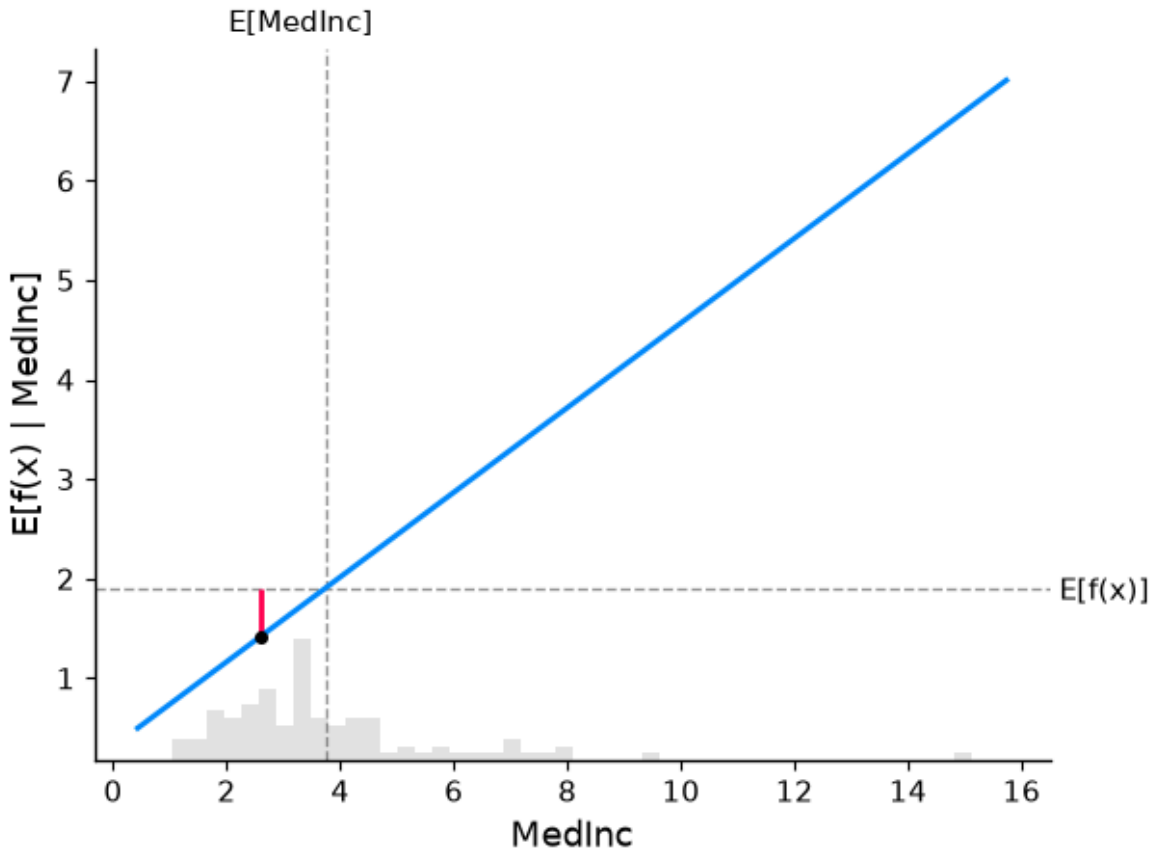
A feature's importance in a model depends on both how it affects the model's output and the distribution of its values, which can be visualized together using a partial dependence plot with a feature value histogram on the x-axis.

In [5]:
```
shap.partial_dependence_plot(
    "MedInc",
    model.predict,
    X100,
    ice=False,
    model_expected_value=True,
    feature_expected_value=True,
)
```

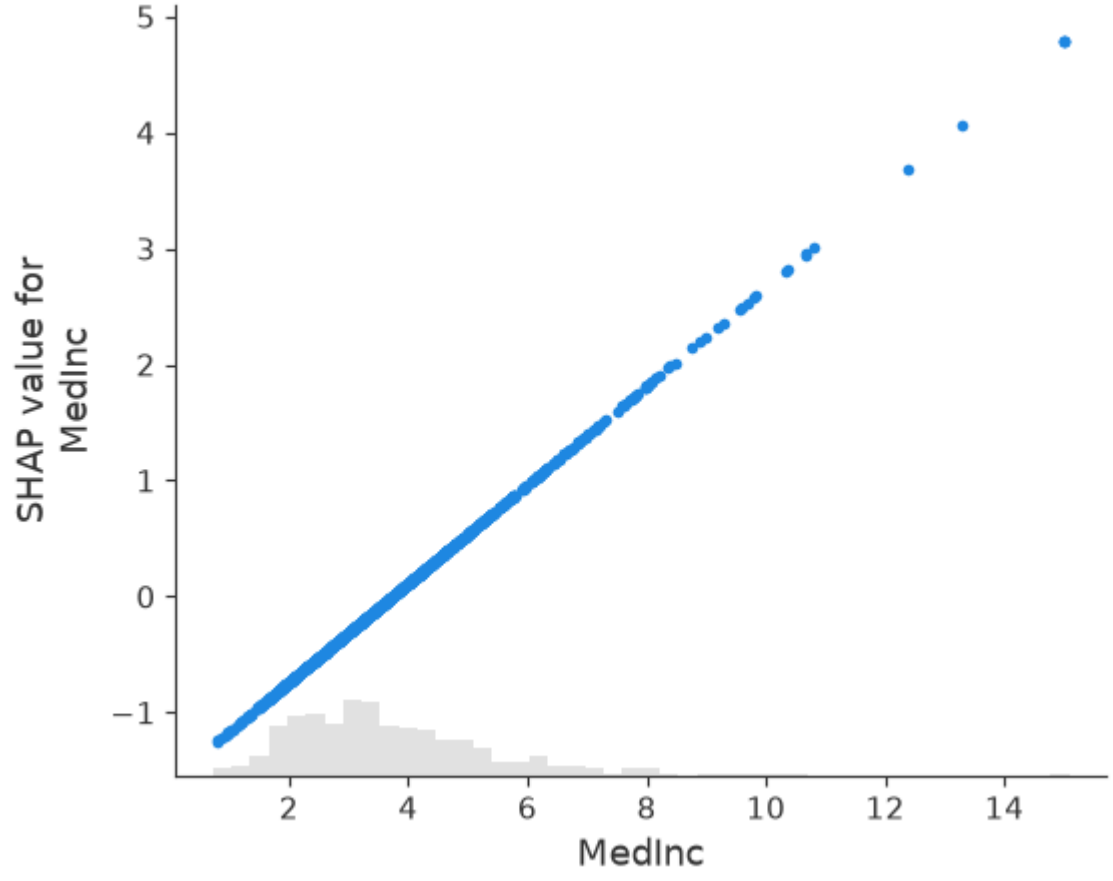


In [6]:
```python
# compute the SHAP values for the linear model
explainer = shap.Explainer(model.predict, X100)
shap_values = explainer(X)

# make a standard partial dependence plot
sample_ind = 20
shap.partial_dependence_plot(
    "MedInc",
    model.predict,
    X100,
    model_expected_value=True,
    feature_expected_value=True,
    ice=False,
    shap_values=shap_values[sample_ind : sample_ind + 1, :],
)
```

```
ExactExplainer explainer: 1001it [00:12, 52.99it/s]
```



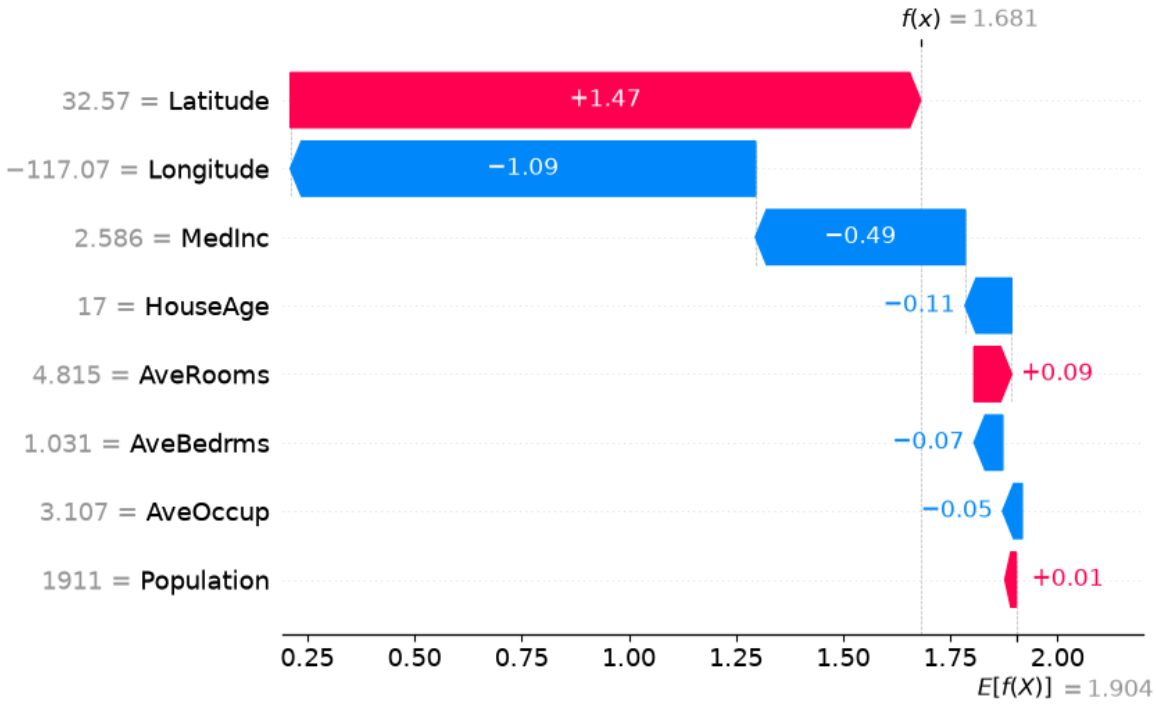
In [7]:
```
shap.plots.scatter(shap_values[:, "MedInc"])
```



The additive nature of Shapley values

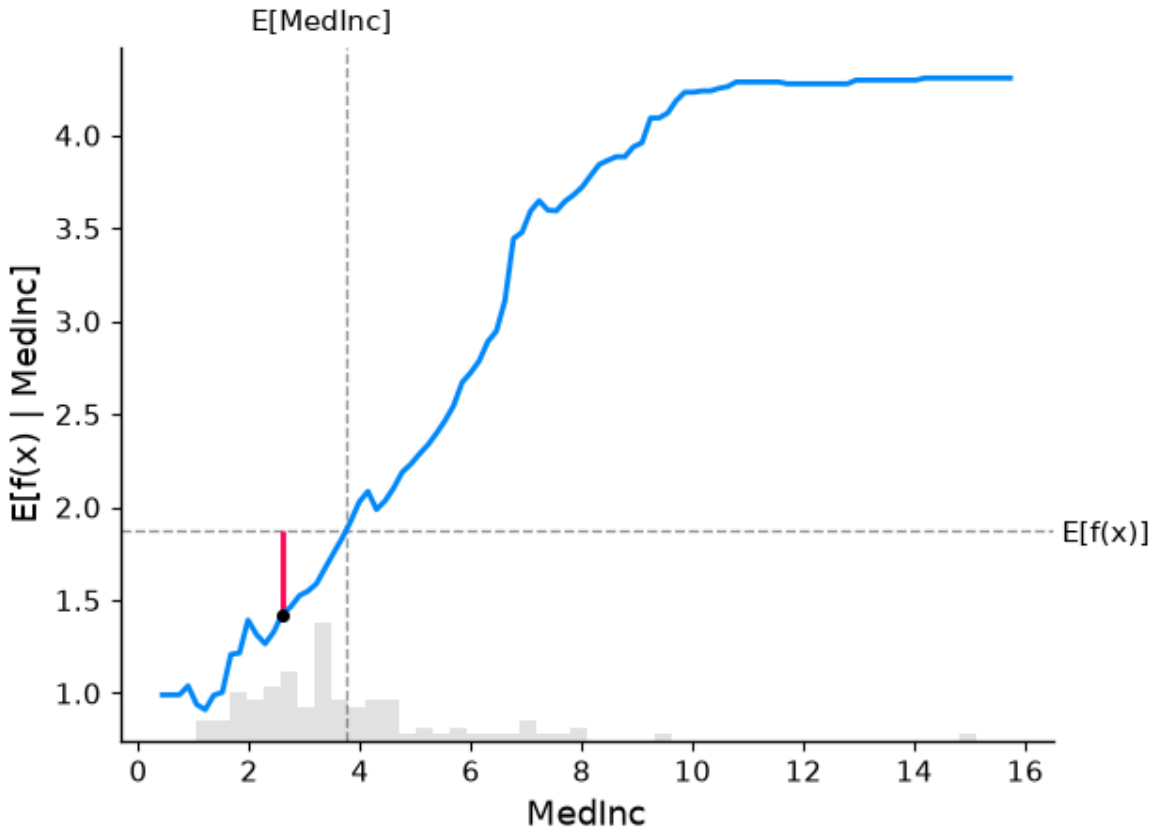
SHAP values always sum to the difference between the baseline (expected) model output and the actual prediction, which is best visualized through a waterfall plot that adds each feature's contribution one at a time.

In [8]:
```
# the waterfall_plot shows how we get from shap_values.base_values to model.pred
shap.plots.waterfall(shap_values[sample_ind], max_display=14)
```

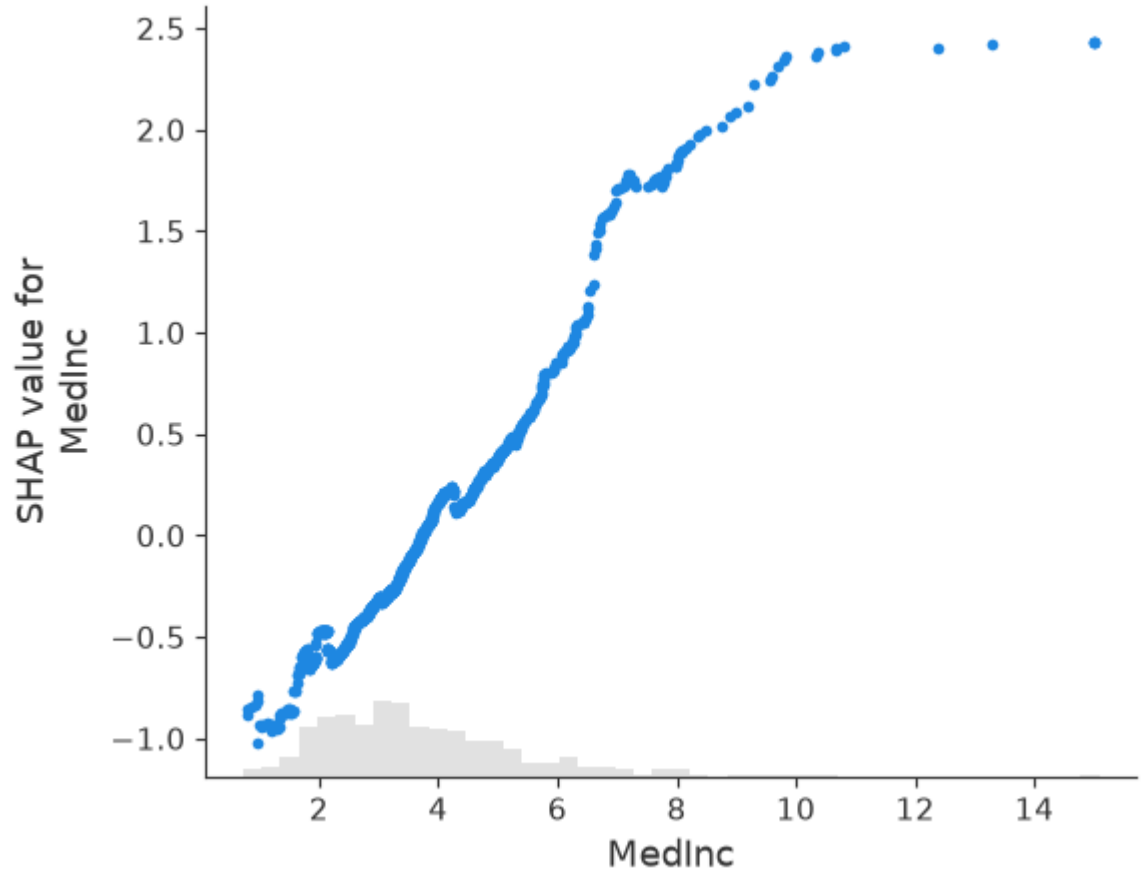


In [10]:
```
# fit a GAM model to the data
import interpret.glassbox

model_ebm = interpret.glassbox.ExplainableBoostingRegressor(interactions=0)
model_ebm.fit(X, y)

# explain the GAM model with SHAP
explainer_ebm = shap.Explainer(model_ebm.predict, X100)
shap_values_ebm = explainer_ebm(X)

# make a standard partial dependence plot with a single SHAP value overlaid
fig, ax = shap.partial_dependence_plot(
    "MedInc",
    model_ebm.predict,
    X100,
    model_expected_value=True,
    feature_expected_value=True,
    show=False,
    ice=False,
    shap_values=shap_values_ebm[sample_ind : sample_ind + 1, :],
)
```

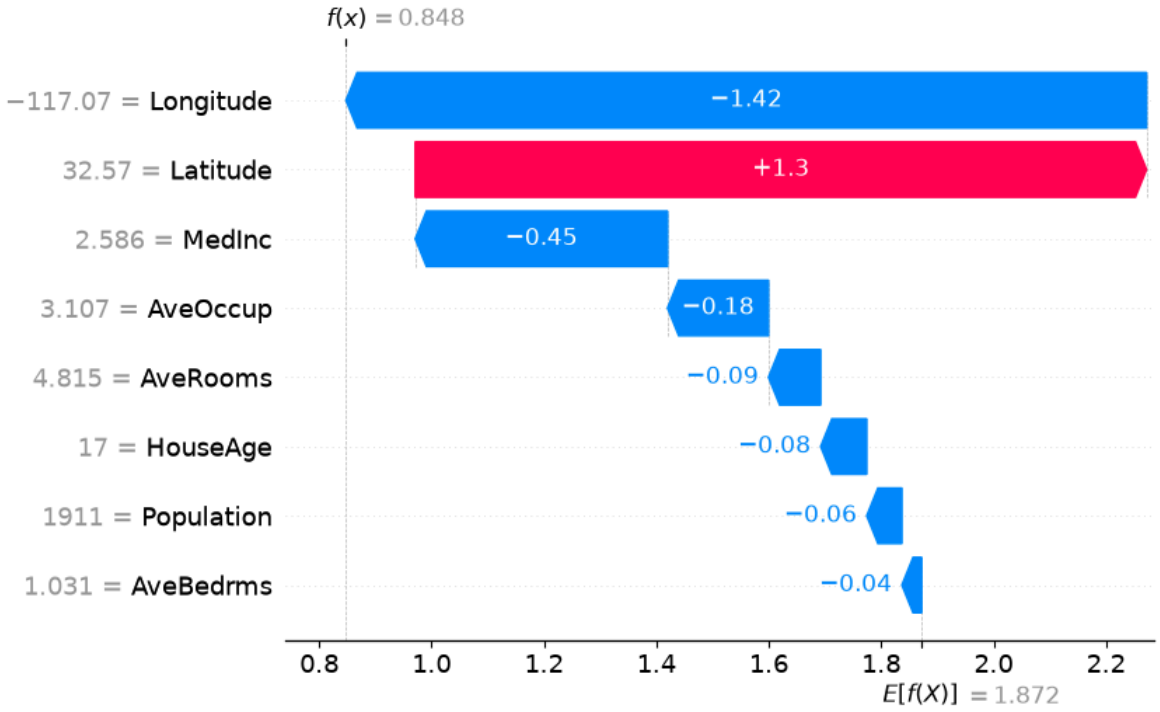
```
ExactExplainer explainer: 1001it [00:25, 23.97it/s]
```



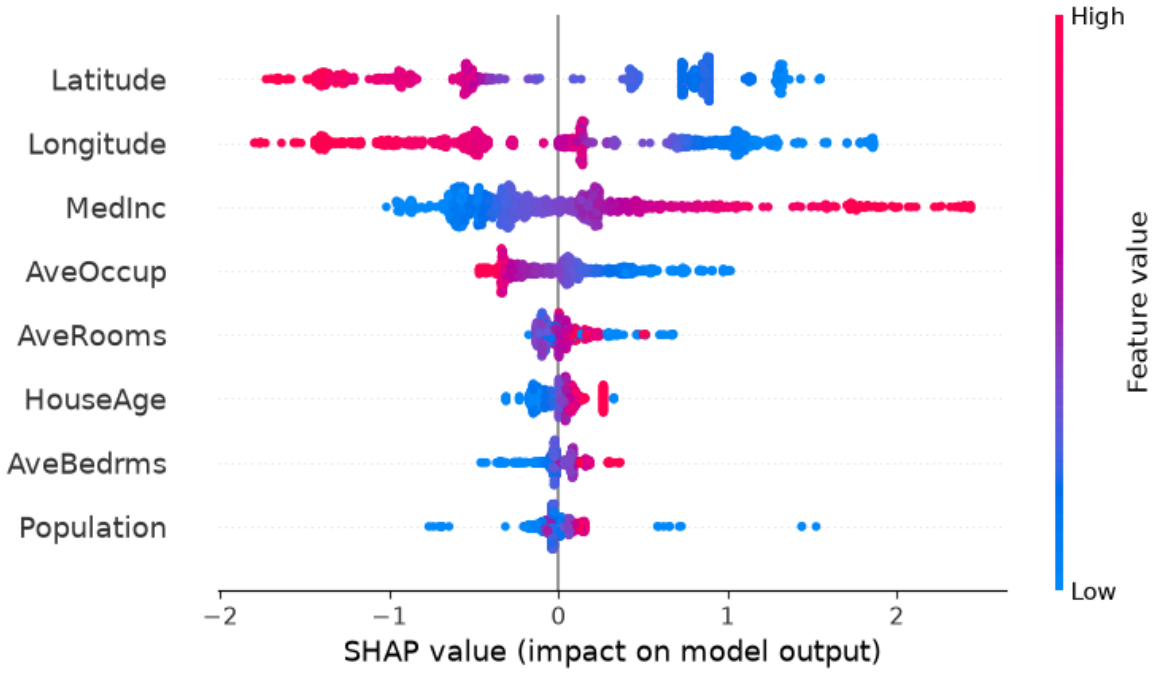
In [11]:
```
shap.plots.scatter(shap_values_ebm[:, "MedInc"])
```



In [12]:
```
# the waterfall_plot shows how we get from explainer.expected_value to model.pre
shap.plots.waterfall(shap_values_ebm[sample_ind])
```



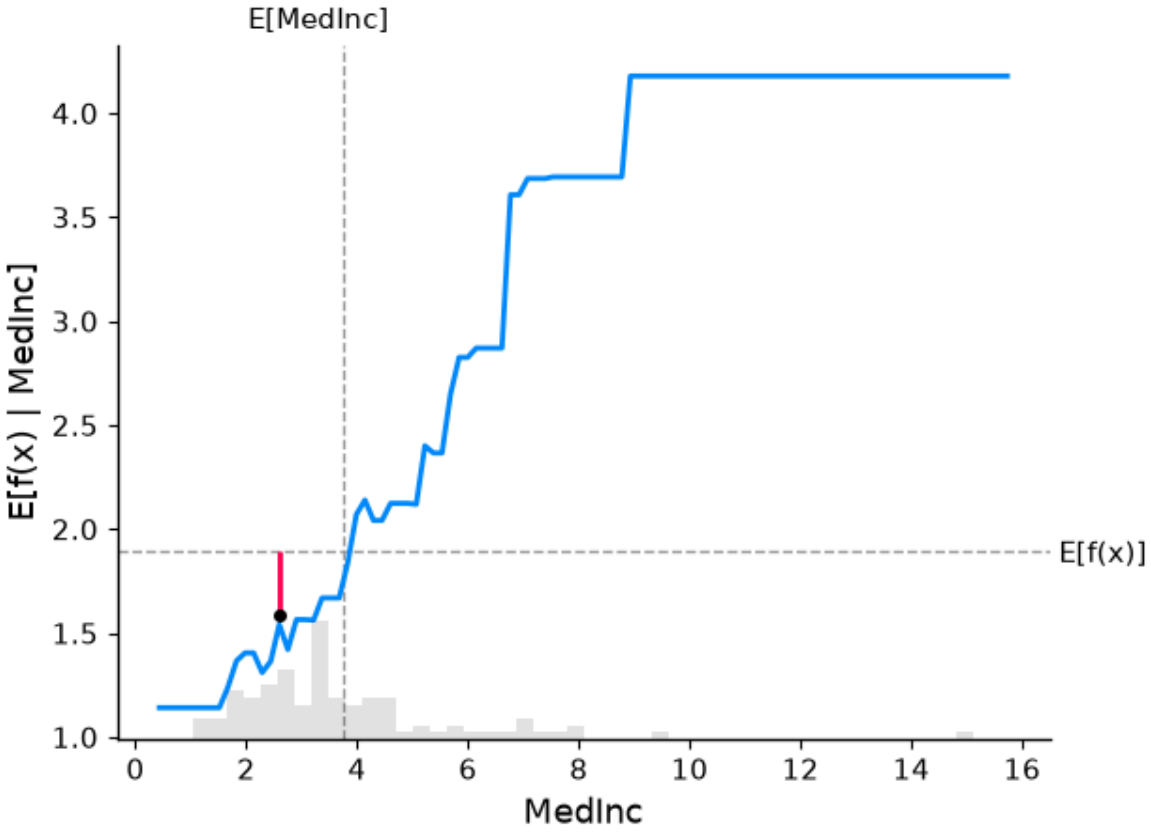
In [13]:
```
# the beeswarm plot displays SHAP values for each feature across all examples,
# with colors indicating how the SHAP values correlate with feature values
shap.plots.beeswarm(shap_values_ebm)
```



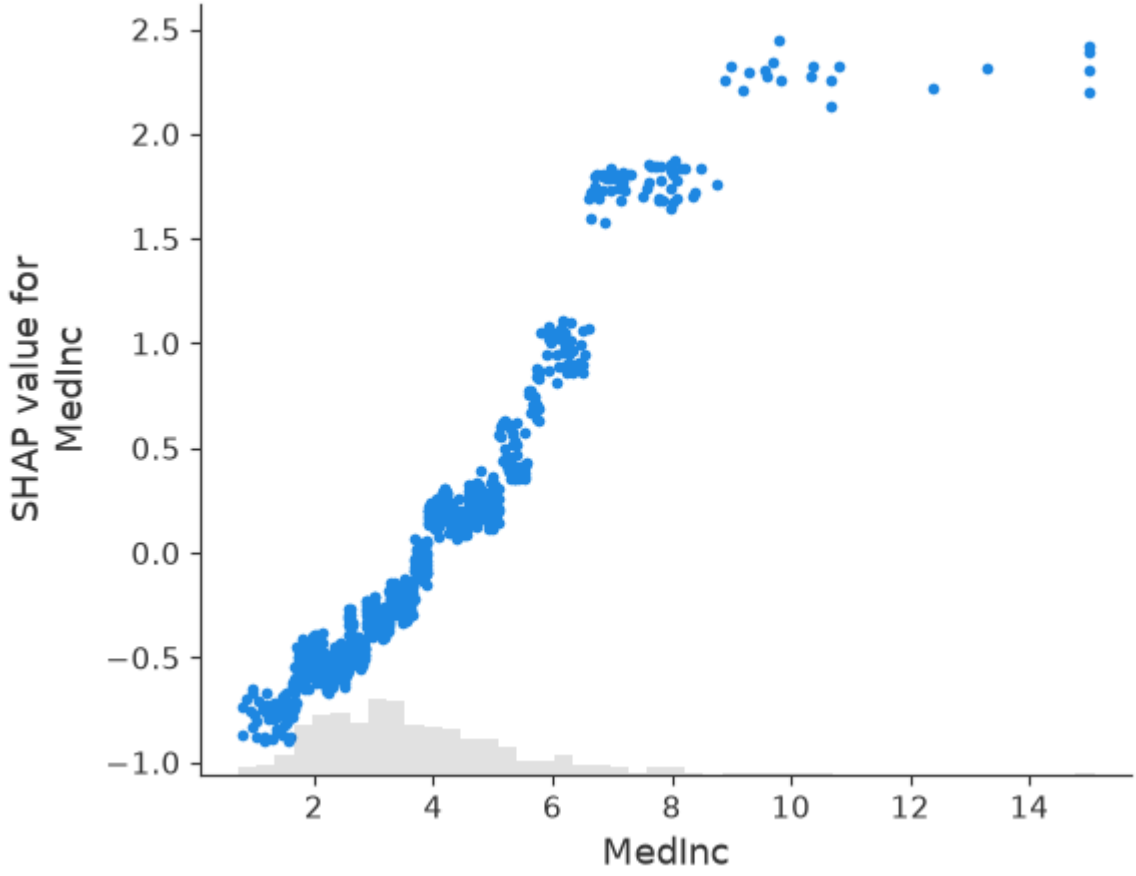
Explaining a non-additive boosted tree model

In [15]:
```
# train XGBoost model
import xgboost

model_xgb = xgboost.XGBRegressor(n_estimators=100, max_depth=2).fit(X, y)

# explain the GAM model with SHAP
explainer_xgb = shap.Explainer(model_xgb, X100)
shap_values_xgb = explainer_xgb(X)
```

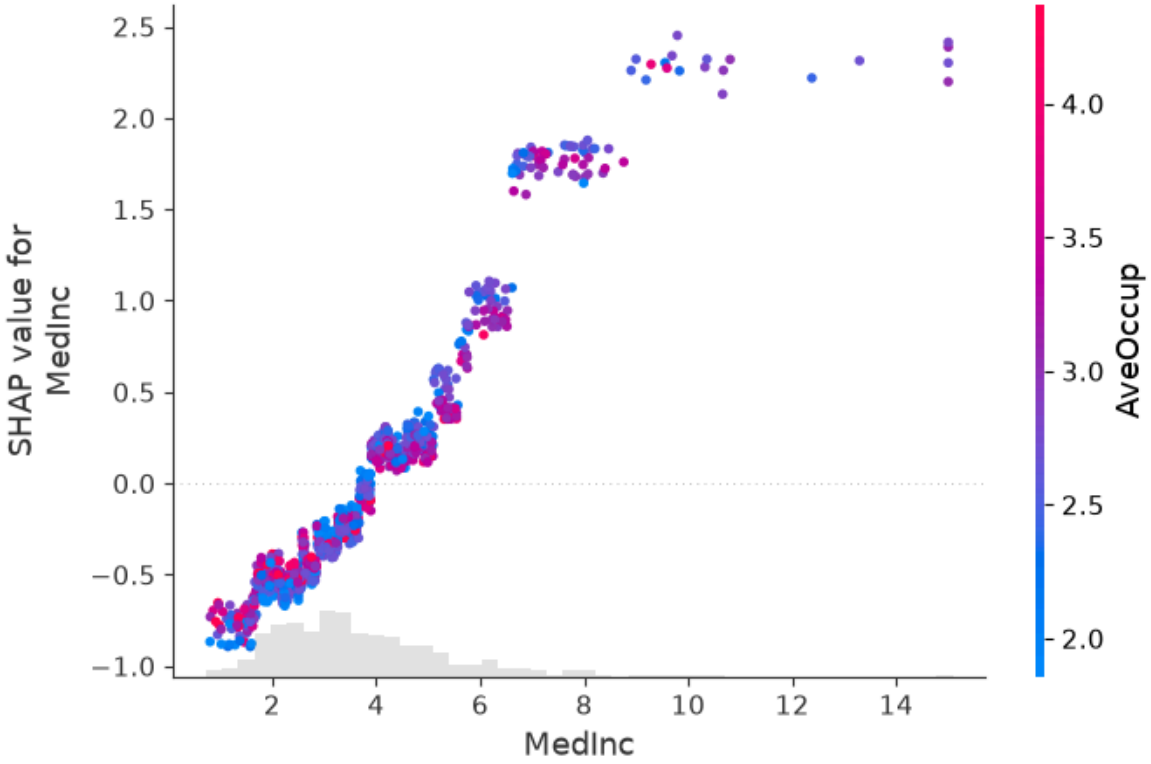
```
# make a standard partial dependence plot with a single SHAP value overlaid
fig, ax = shap.partial_dependence_plot(
    "MedInc",
    model_xgb.predict,
    X100,
    model_expected_value=True,
    feature_expected_value=True,
    show=False,
    ice=False,
    shap_values=shap_values_xgb[sample_ind : sample_ind + 1, :],
)
```



In [18]:
```
shap.plots.scatter(shap_values_xgb[:, "MedInc"])
```



In [22]:
```
shap.plots.scatter(shap_values_xgb[:, "MedInc"], color=shap_values)
```

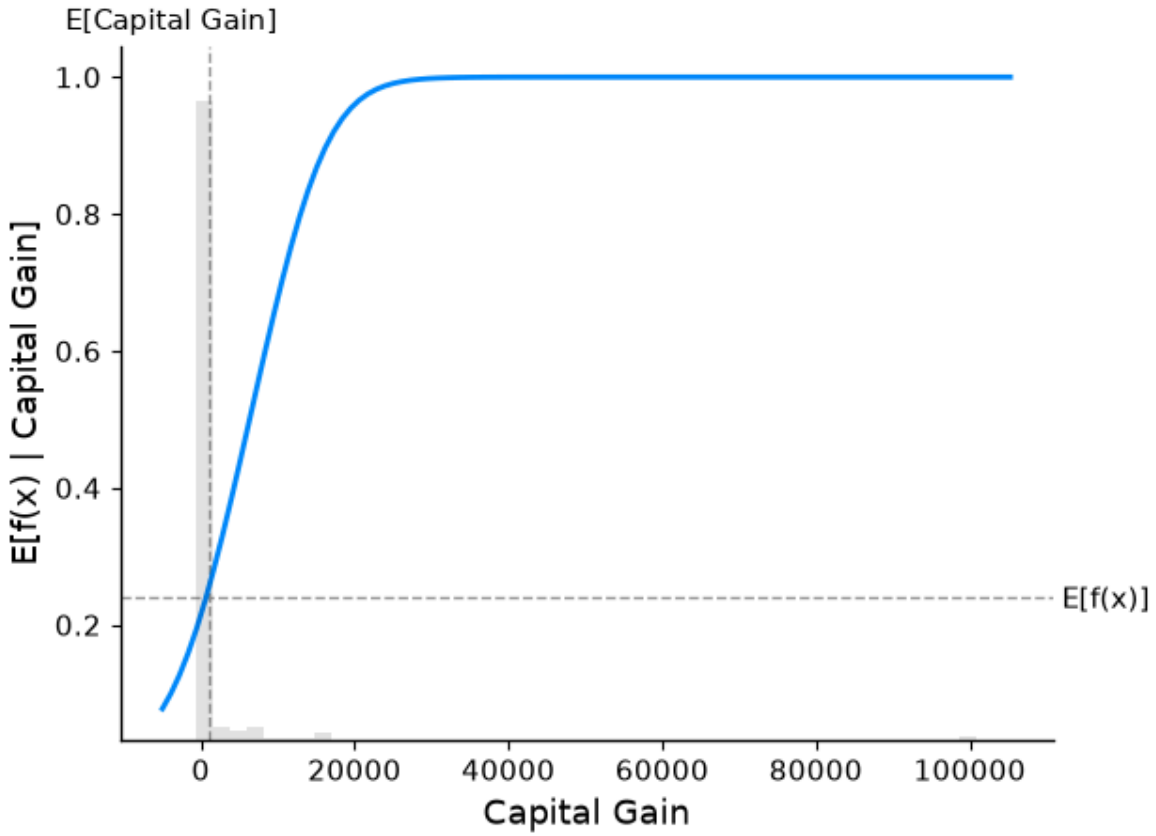


Explaining a linear logistic regression model

In [25]:
```
# a classic adult census dataset price dataset
X_adult, y_adult = shap.datasets.adult()
```

```
# a simple linear logistic model
model_adult = sklearn.linear_model.LogisticRegression(max_iter=10000)
model_adult.fit(X_adult, y_adult)


def model_adult_proba(x):
    return model_adult.predict_proba(x)[:, 1]


def model_adult_log_odds(x):
    p = model_adult.predict_log_proba(x)
    return p[:, 1] - p[:, 0]
```

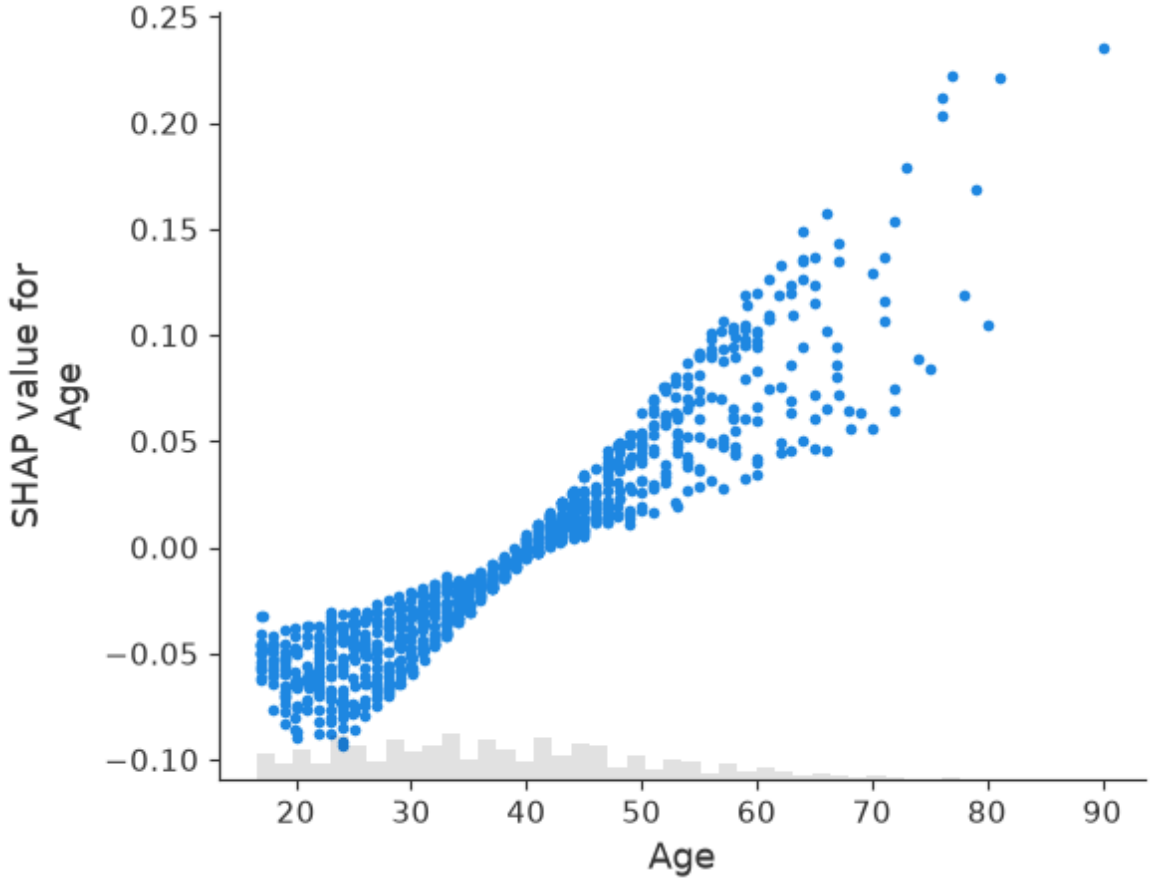
In [26]:
```
# make a standard partial dependence plot
sample_ind = 18
fig, ax = shap.partial_dependence_plot(
    "Capital Gain",
    model_adult_proba,
    X_adult,
    model_expected_value=True,
    feature_expected_value=True,
    show=False,
    ice=False,
)
```



In [27]:
```
# compute the SHAP values for the linear model
background_adult = shap.maskers.Independent(X_adult, max_samples=100)
explainer = shap.Explainer(model_adult_proba, background_adult)
shap_values_adult = explainer(X_adult[:1000])
```

```
PermutationExplainer explainer: 1001it [01:07, 13.59it/s]
```

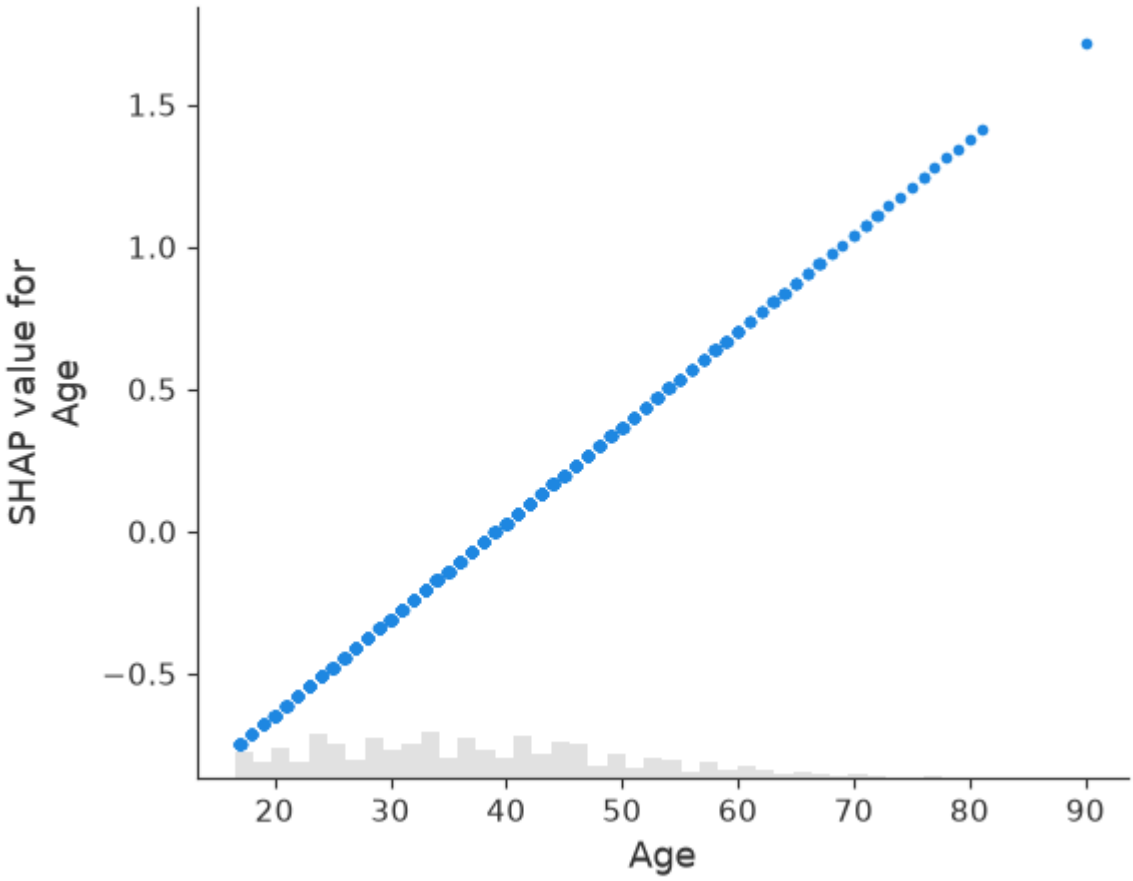
In [28]:
```
shap.plots.scatter(shap_values_adult[:, "Age"])
```



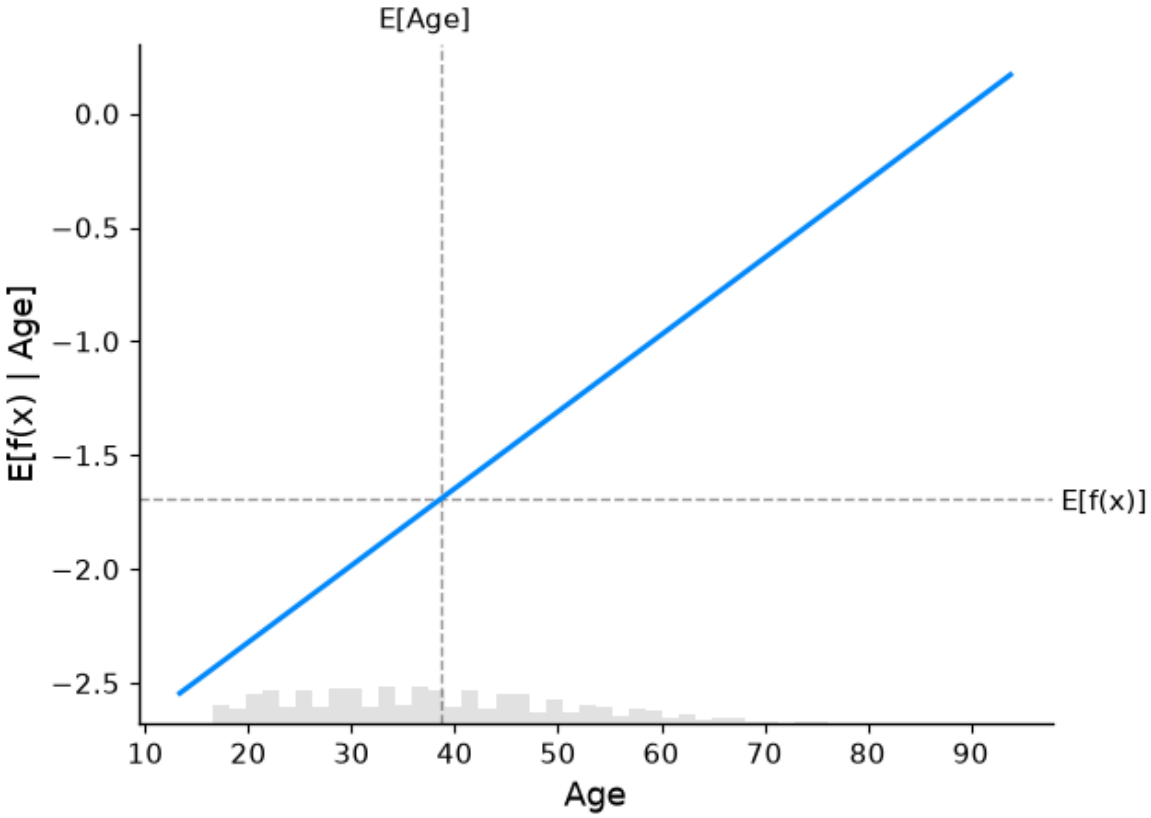
In [30]:
```
# compute the SHAP values for the linear model
explainer_log_odds = shap.Explainer(model_adult_log_odds, background_adult)
shap_values_adult_log_odds = explainer_log_odds(X_adult[:1000])
```

```
PermutationExplainer explainer: 1001it [00:53, 15.27it/s]
```

In [31]:
```
shap.plots.scatter(shap_values_adult_log_odds[:, "Age"])
```



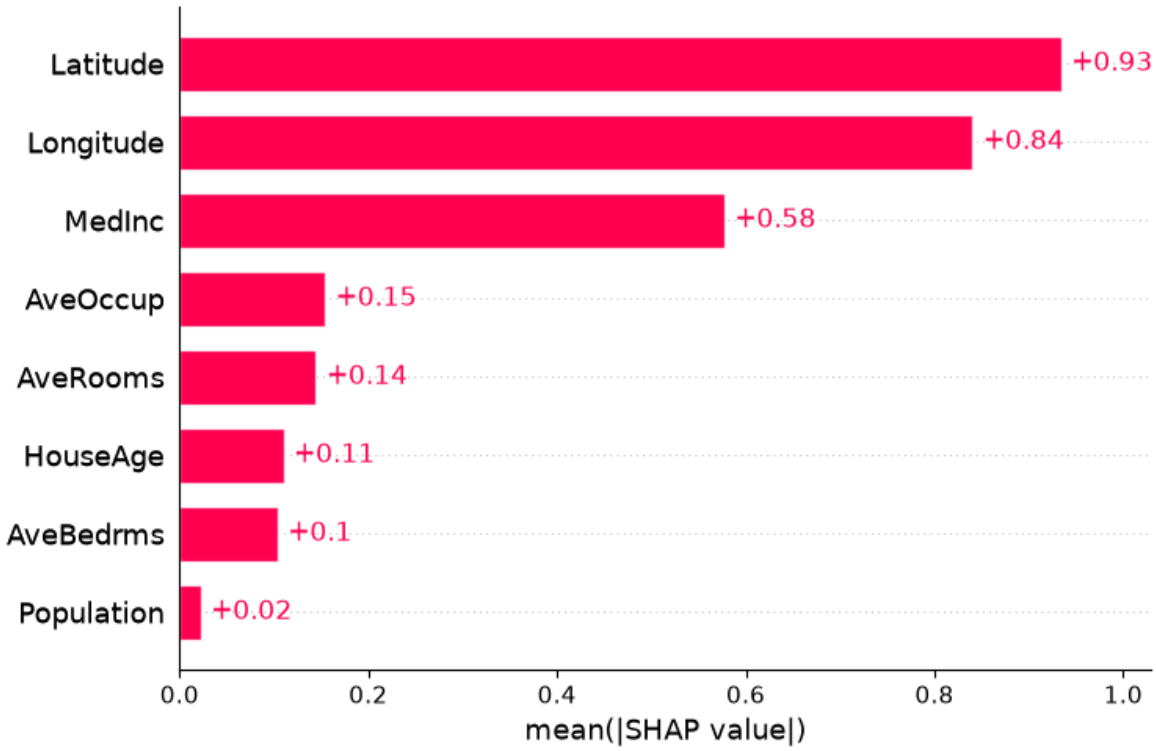
In [32]:
```
# make a standard partial dependence plot
sample_ind = 18
fig, ax = shap.partial_dependence_plot(
    "Age",
    model_adult_log_odds,
    X_adult,
    model_expected_value=True,
    feature_expected_value=True,
    show=False,
    ice=False,
)
```



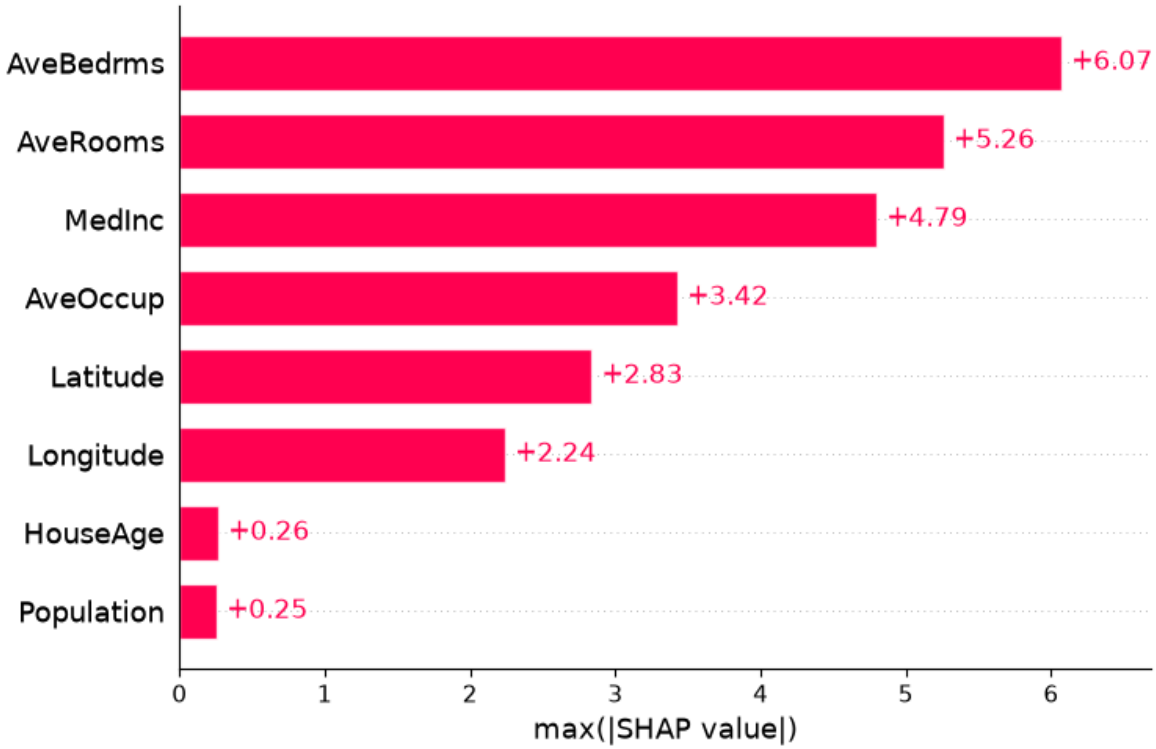
Explaining a non-additive boosted tree logistic regression model

In [39]:
```
# train XGBoost model
model = xgboost.XGBClassifier(
    n_estimators=100,
    max_depth=2,
    eval_metric="logloss"   # <-- Move it here, into the constructor
).fit(X_adult, y_adult * 1)

# compute SHAP values
explainer = shap.Explainer(model, background_adult)
```

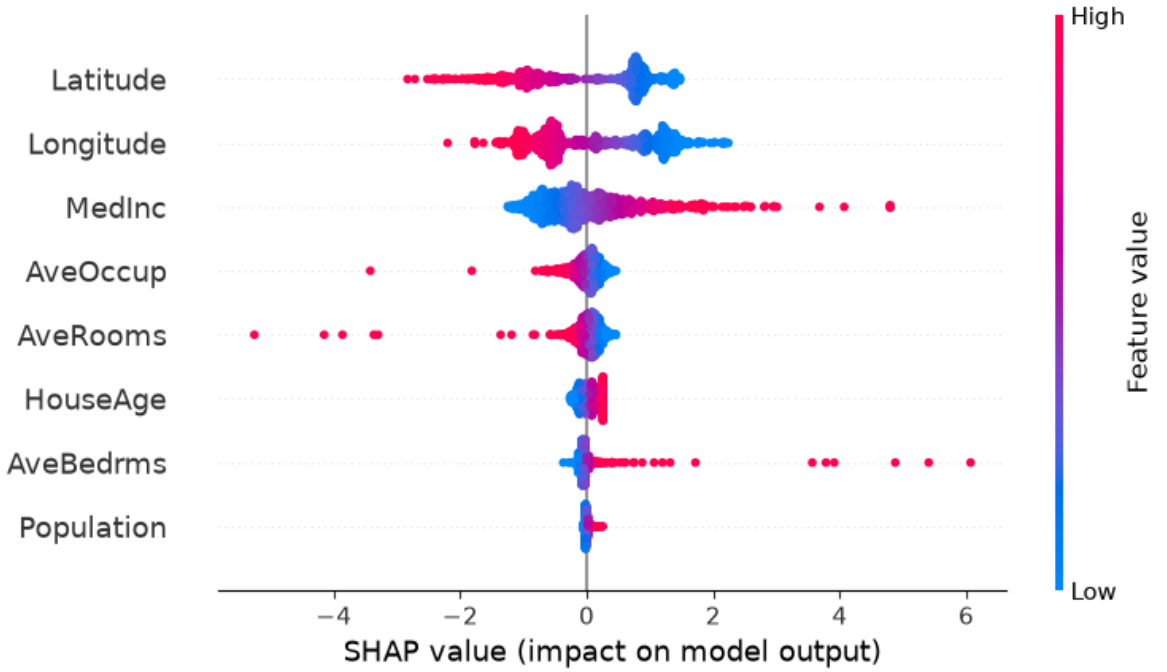
In [36]:
```
shap.plots.bar(shap_values)
```



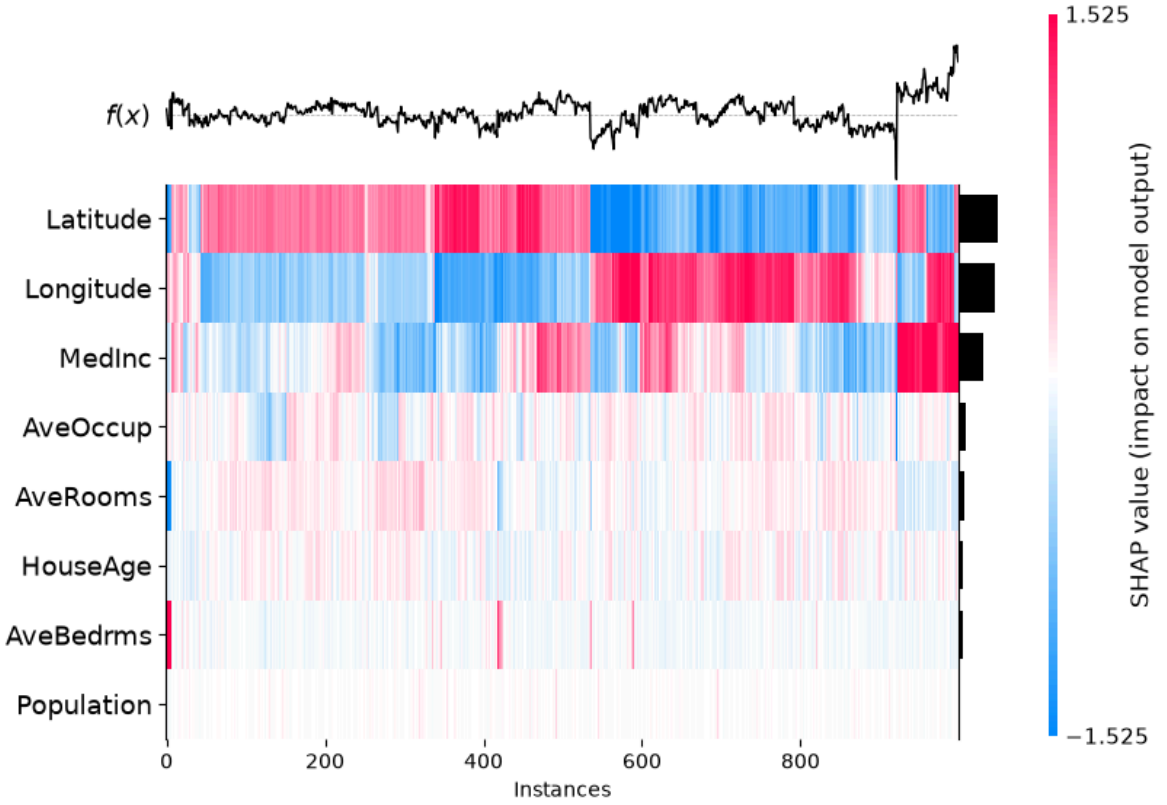
In [37]:
```
shap.plots.bar(shap_values.abs.max(0))
```



In [38]:
```
shap.plots.beeswarm(shap_values)
```



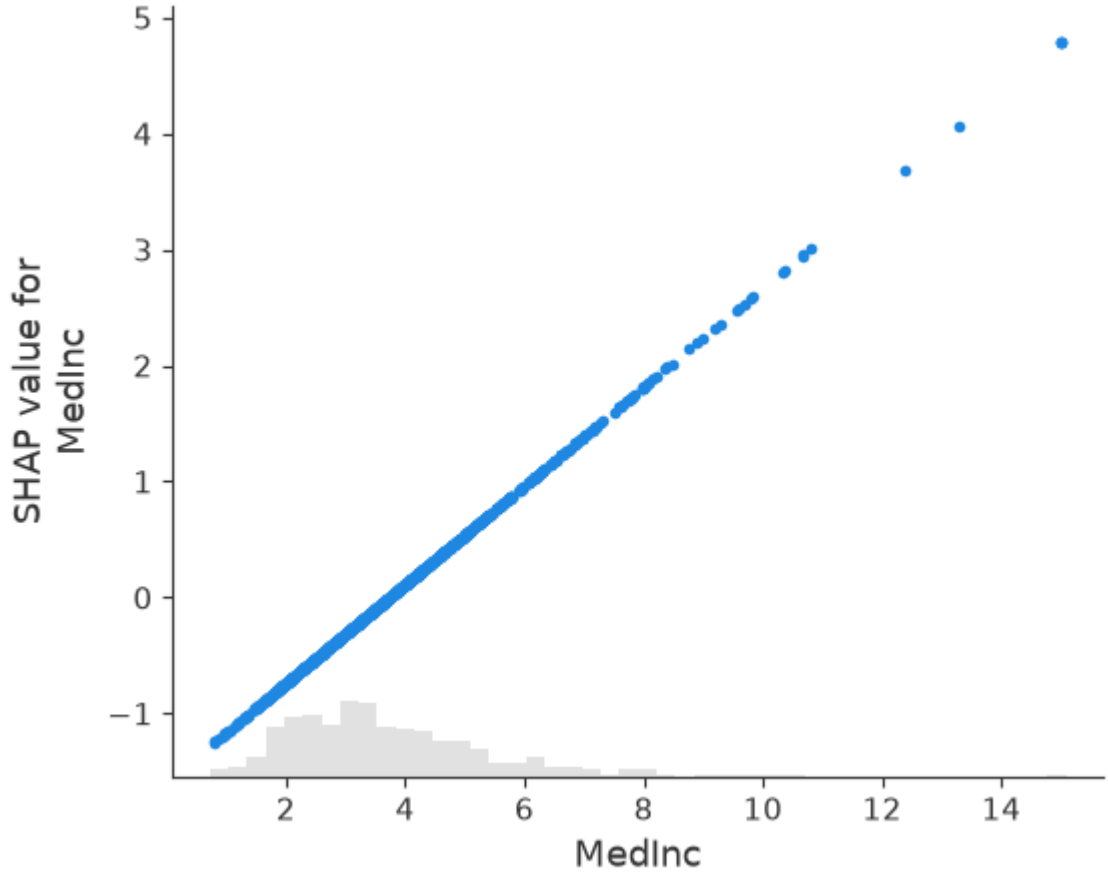
In [40]:
```
shap.plots.heatmap(shap_values[:1000])
```



Out[40]: <Axes: xlabel='Instances'>

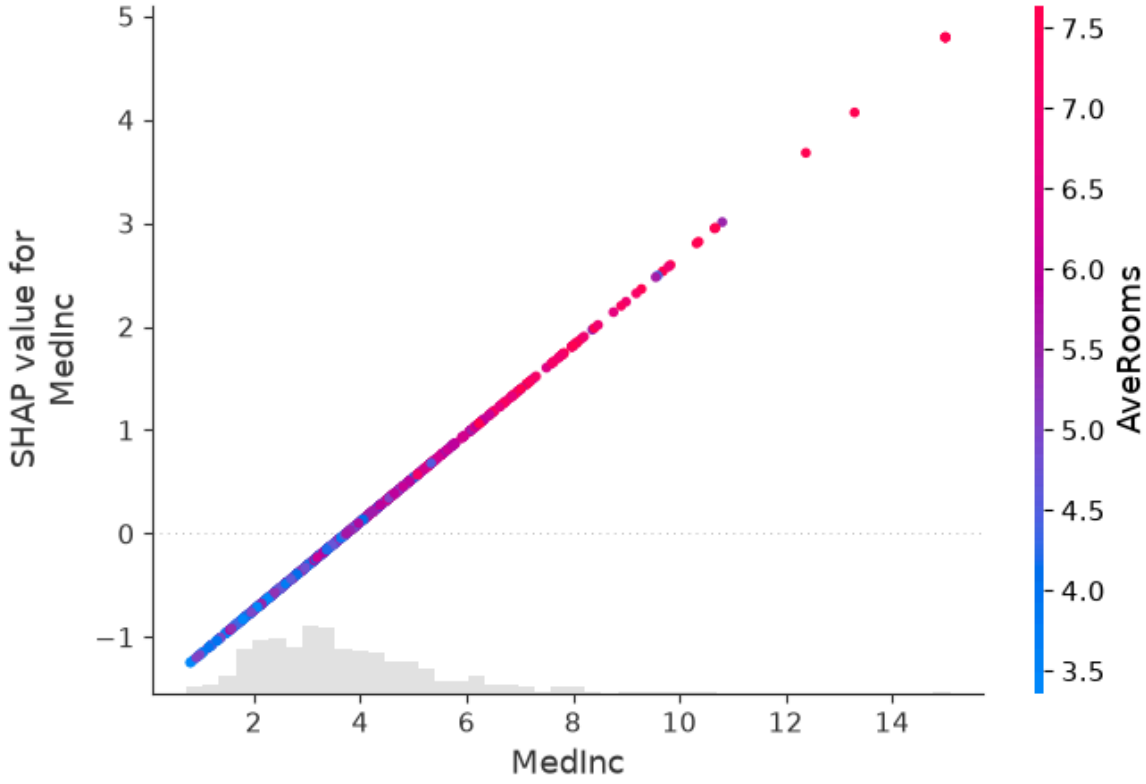
In [42]:
```
# Check column index
age_index = list(X_adult.columns).index("Age")
print(age_index)  # e.g., 0

# Use the index instead of the name
shap.plots.scatter(shap_values[:, age_index])
```

0



In [45]:
```
# Check column index
age_index = list(X_adult.columns).index("Age")
print(age_index)  # e.g., 0

# Use the index instead of the name
shap.plots.scatter(shap_values[:, age_index], color=shap_values)
```

```
0
```



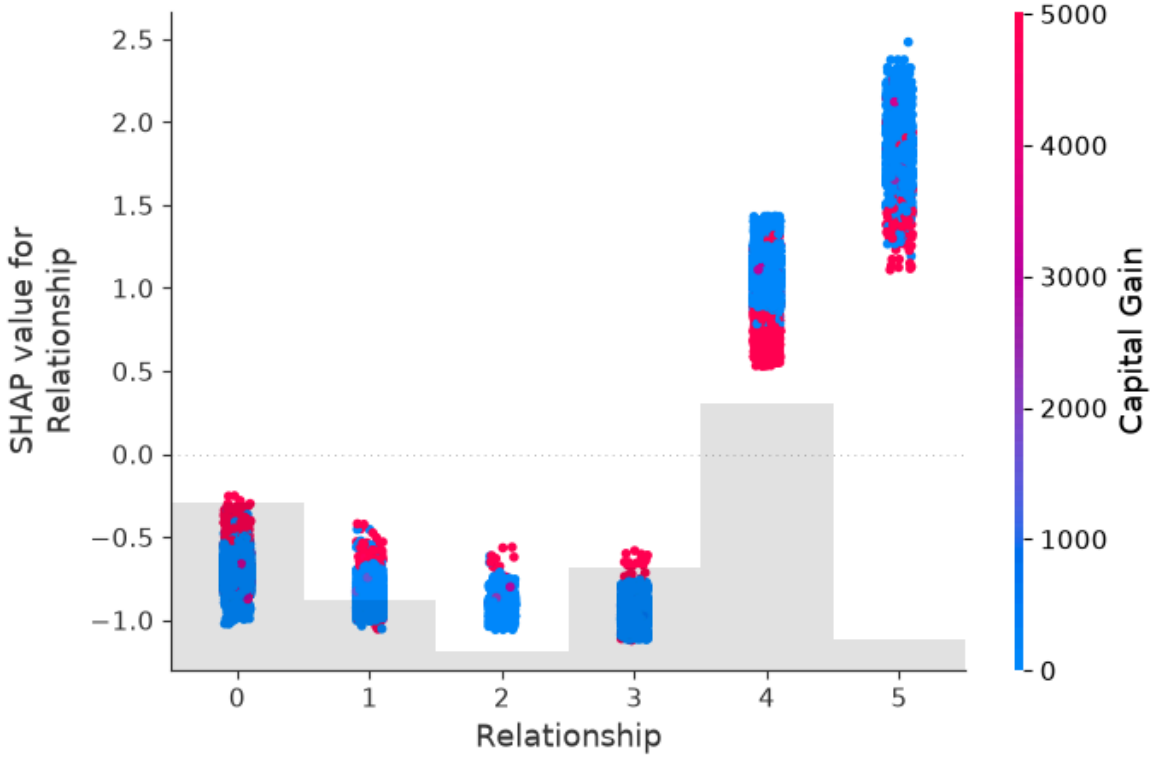
In [50]:
```
# Check number of features in each
print("X_adult columns:", list(X_adult.columns))
print("X_adult shape:", X_adult.shape)

print("shap_values shape:", shap_values.shape)
print("shap_values feature_names:", shap_values.feature_names)
```

```
X_adult columns: ['Age', 'Workclass', 'Education-Num', 'Marital Status', 'Occupat
ion', 'Relationship', 'Race', 'Sex', 'Capital Gain', 'Capital Loss', 'Hours per w
eek', 'Country']
X_adult shape: (32561, 12)
shap_values shape: (1000, 8)
shap_values feature_names: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Popul
ation', 'AveOccup', 'Latitude', 'Longitude']
```

In [51]:
```
import pandas as pd

# Make sure X_adult is a proper DataFrame with column names
# (replace feature_names with your actual column list if needed)
print(X_adult.columns.tolist())  # confirm column names

# Recompute explainer and shap values
explainer = shap.Explainer(model, background_adult)
shap_values = explainer(X_adult)  # X_adult must be a DataFrame

# Confirm feature names are now attached
print("Feature names in shap_values:", shap_values.feature_names)
```

```
['Age', 'Workclass', 'Education-Num', 'Marital Status', 'Occupation', 'Relationsh
ip', 'Race', 'Sex', 'Capital Gain', 'Capital Loss', 'Hours per week', 'Country']
 98%|===================| 31815/32561 [00:27<00:00]
Feature names in shap_values: ['Age', 'Workclass', 'Education-Num', 'Marital Stat
us', 'Occupation', 'Relationship', 'Race', 'Sex', 'Capital Gain', 'Capital Loss',
'Hours per week', 'Country']
```

In [52]:
```
shap.plots.scatter(shap_values[:, "Relationship"], color=shap_values)
```

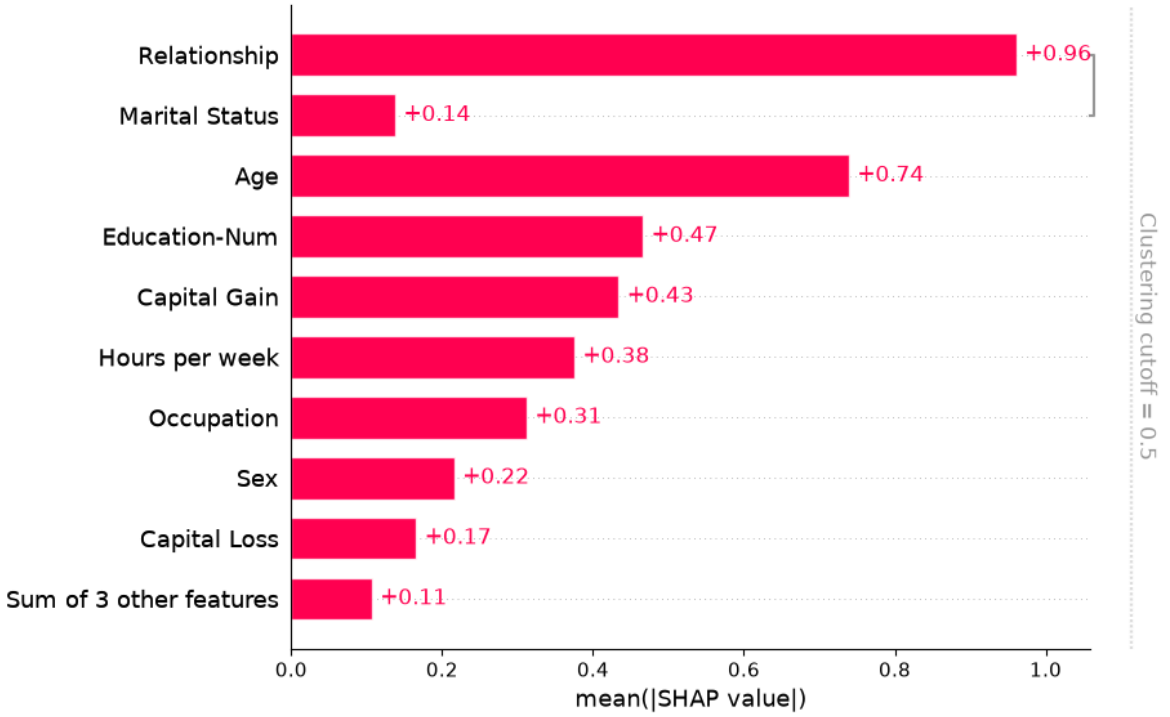


Dealing with correlated features

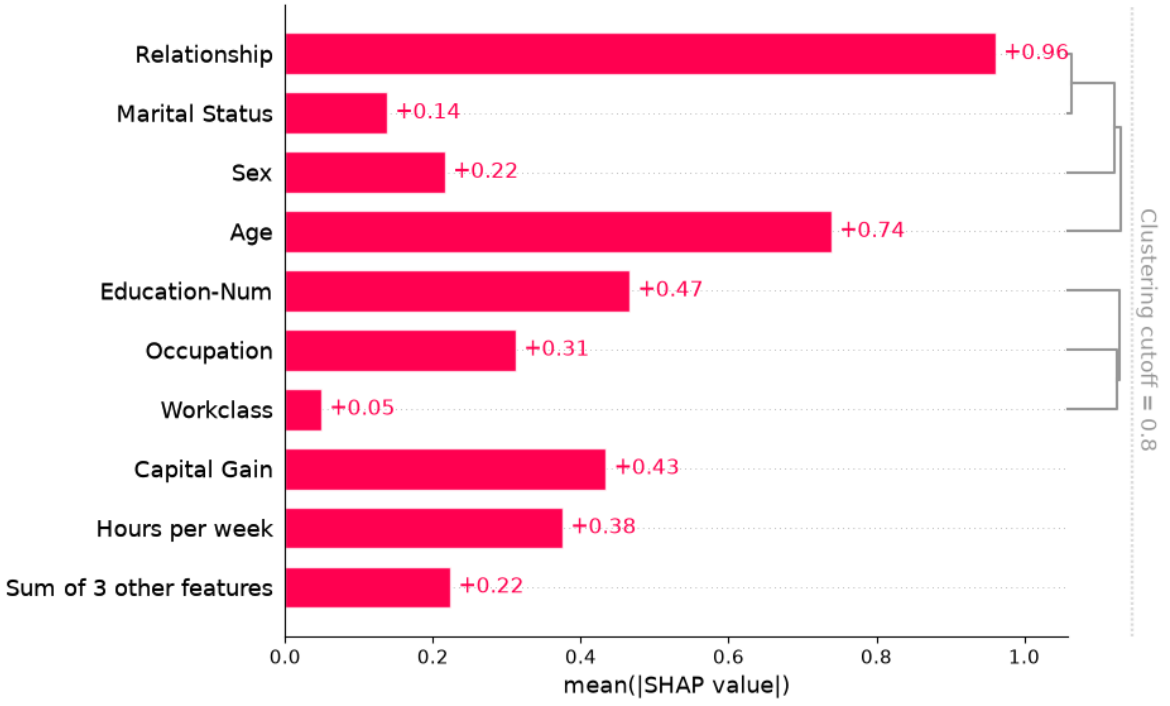
In [53]:
```
clustering = shap.utils.hclust(X_adult, y_adult)
```

```
145it [00:10,  1.51s/it]
```

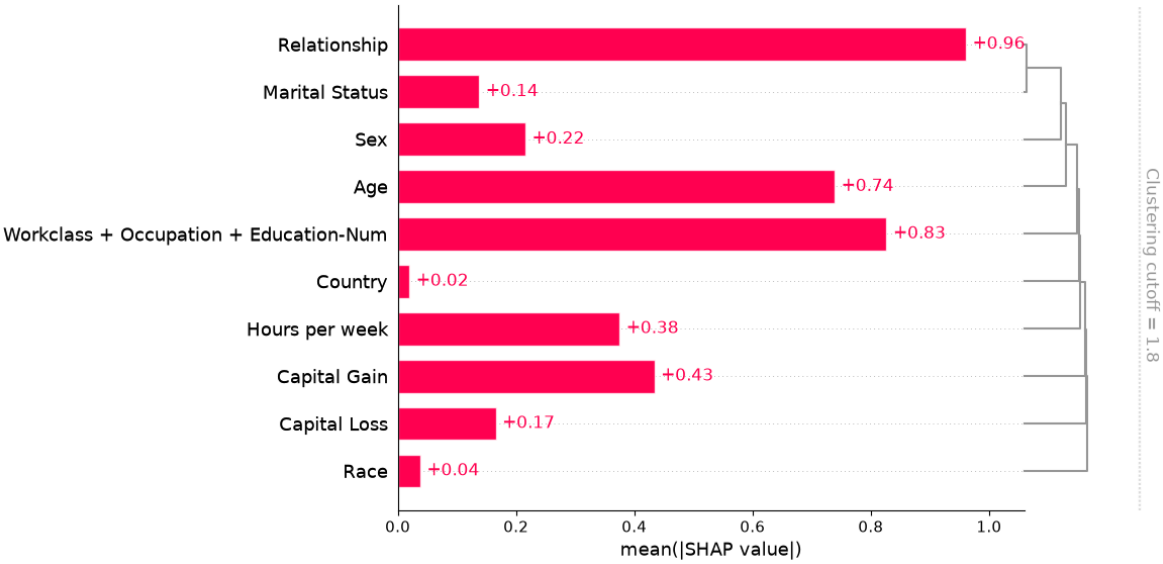
In [54]:
```
shap.plots.bar(shap_values, clustering=clustering)
```



In [55]:
```
shap.plots.bar(shap_values, clustering=clustering, clustering_cutoff=0.8)
```



In [56]:
```
shap.plots.bar(shap_values, clustering=clustering, clustering_cutoff=1.8)
```



Explaining a transformers NLP model

This demonstrates how SHAP can be applied to complex model types with highly structured inputs.

In [62]:

```
# ============================================================
# INSTALL REQUIRED PACKAGES (Run once in terminal)
# ============================================================
# pip install shap transformers datasets torch

# ============================================================
# IMPORTS
# ============================================================
import warnings
warnings.filterwarnings("ignore")

import shap
import datasets
import transformers
import torch
import numpy as np

print("✅ All libraries imported successfully!")
print(f"   SHAP version      : {shap.__version__}")
print(f"   Transformers      : {transformers.__version__}")
print(f"   Datasets          : {datasets.__version__}")
print(f"   Torch             : {torch.__version__}")

# ============================================================
# STEP 1: LOAD TOKENIZER & MODEL
# ============================================================
MODEL_NAME = "distilbert-base-uncased-finetuned-sst-2-english"

print("\n🔄 Loading tokenizer and model...")

tokenizer = transformers.AutoTokenizer.from_pretrained(
    MODEL_NAME,
    use_fast=True
)
```

```
model = transformers.AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME
)
model.eval()  # Set model to evaluation mode

print("✅ Model and tokenizer loaded!")

# ============================================================
# STEP 2: DEFINE PREDICTION FUNCTION FOR SHAP
# ============================================================
def f(x):
    """
    Prediction function for SHAP explainer.
    Takes list of strings, returns logits as numpy array.
    """
    # Handle both list and numpy array inputs
    if isinstance(x, np.ndarray):
        x = x.tolist()

    # Tokenize inputs
    tv = tokenizer(
        x,
        padding=True,
        truncation=True,
        max_length=512,
        return_tensors="pt"
    )

    # Get model predictions
    with torch.no_grad():
        outputs = model(**tv)

    # Return logits as numpy
    return outputs.logits.detach().numpy()

print("✅ Prediction function defined!")

# ============================================================
# STEP 3: CREATE SHAP EXPLAINER
# ============================================================
print("\n🔄 Creating SHAP explainer...")

explainer = shap.Explainer(
    f,
    tokenizer,
    output_names=["NEGATIVE", "POSITIVE"]  # SST-2 labels
)

print("✅ SHAP Explainer created!")

# ============================================================
# STEP 4: LOAD IMDB DATASET (With Fallback Options)
# ============================================================
print("\n🔄 Loading IMDB dataset...")

imdb_train = None

# --- Attempt 1: New HuggingFace namespace format ---
try:
```

```
    imdb_dataset = datasets.load_dataset(
        "stanfordnlp/imdb",
        trust_remote_code=True
    )
    imdb_train = imdb_dataset["train"]
    print("✅ Dataset loaded via 'stanfordnlp/imdb'")

except Exception as e1:
    print(f"⚠️  Attempt 1 failed: {e1}")

    # --- Attempt 2: Try alternate source ---
    try:
        imdb_dataset = datasets.load_dataset(
            "ajaykarthick/imdb-movie-reviews",
            trust_remote_code=True
        )
        imdb_train = imdb_dataset["train"]
        print("✅ Dataset loaded via alternate source!")

    except Exception as e2:
        print(f"⚠️  Attempt 2 failed: {e2}")

        # --- Attempt 3: Use manual sample texts (Offline fallback) ---
        print("⚠️  Using built-in sample texts as fallback...")

        sample_texts = [
            "This movie was absolutely fantastic! The acting was superb and the
            "Terrible film. Waste of time and money. Poorly directed and badly a
            "An outstanding masterpiece. One of the best films I have ever seen
            "Boring and predictable. Nothing new to offer. Completely disappoint
            "Great performances from the entire cast. The cinematography was bre
            "Awful screenplay with no character development. The plot made absol
            "Loved every minute of it! Highly recommended to everyone who enjoys
            "One of the worst movies ever made. Completely unwatchable and a tot
            "Brilliant direction and superb music. The emotional scenes were inc
            "I fell asleep halfway through. Extremely dull with no interesting m
        ]

        # Create a simple dataset-like structure
        class SimpleDataset:
            def __init__(self, texts):
                self.data = {"text": texts}

            def __getitem__(self, key):
                return self.data[key]

        imdb_train = SimpleDataset(sample_texts)
        print("✅ Using 10 built-in sample reviews!")

# ============================================================
# STEP 5: PREPARE TEXT SAMPLES
# ============================================================
print("\n🔄 Preparing text samples for SHAP analysis...")

# Get 10 text samples safely
if hasattr(imdb_train, '__getitem__'):
    try:
        # For HuggingFace datasets
        texts = imdb_train[:10]["text"]
    except Exception:
```

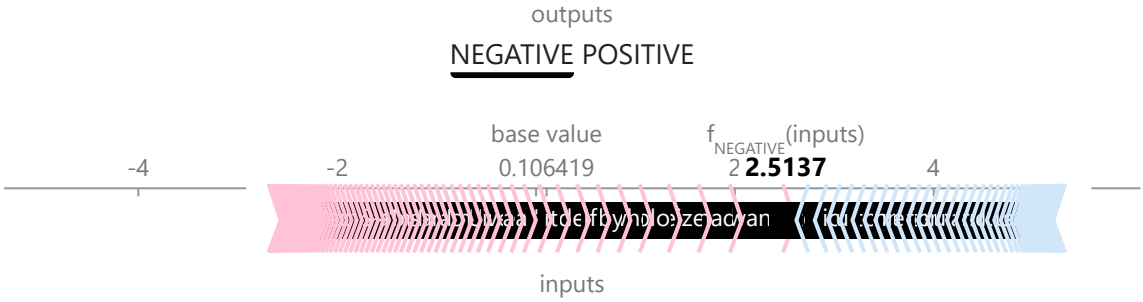
```
        # For custom dataset
        texts = imdb_train["text"][:10]
else:
    texts = list(imdb_train)[:10]

# Ensure all texts are strings and truncate to safe length
texts = [str(t)[:512] for t in texts]

print(f"✅ Prepared {len(texts)} text samples!")
print(f"\n📝 First sample preview:")
print(f"   {texts[0][:150]}...")

# ============================================================
# STEP 6: COMPUTE SHAP VALUES
# ============================================================
print(f"\n🔄 Computing SHAP values for {len(texts)} samples...")
print("   (This may take a few minutes...)\n")

try:
    shap_values = explainer(
        texts,
        fixed_context=1,
        batch_size=2
    )
    print("✅ SHAP values computed successfully!")

except Exception as e:
    print(f"⚠️  Full batch failed: {e}")
    print("🔄 Retrying with smaller batch (3 samples)...")

    shap_values = explainer(
        texts[:3],
        fixed_context=1,
        batch_size=1
    )
    print("✅ SHAP values computed for 3 samples!")

# ============================================================
# STEP 7: VISUALIZE SHAP VALUES
# ============================================================
print("\n🔄 Generating SHAP visualizations...")

# --- Plot 1: Text plot for FIRST review ---
print("\n📊 Plot 1: Token-level SHAP for Review #1")
print("-" * 50)
try:
    shap.plots.text(shap_values[0])
except Exception as e:
    print(f"Text plot error: {e}")
    shap.text_plot(shap_values[0])  # Fallback

# --- Plot 2: Text plot for SECOND review ---
print("\n📊 Plot 2: Token-level SHAP for Review #2")
print("-" * 50)
try:
    shap.plots.text(shap_values[1])
except Exception as e:
    print(f"Text plot error: {e}")

# --- Plot 3: Bar plot - Overall feature importance ---
```

```python
print("\n📊 Plot 3: Feature Importance Bar Chart")
print("-" * 50)
try:
    shap.plots.bar(shap_values)
except Exception as e:
    print(f"Bar plot error: {e}")

# --- Plot 4: Beeswarm plot ---
print("\n📊 Plot 4: Beeswarm Summary Plot")
print("-" * 50)
try:
    shap.plots.beeswarm(shap_values)
except Exception as e:
    print(f"Beeswarm plot skipped: {e}")

# ============================================================
# STEP 8: PRINT PREDICTION SUMMARY
# ============================================================
print("\n" + "="*60)
print("📋 PREDICTION SUMMARY")
print("="*60)

label_map = {0: "NEGATIVE ❌", 1: "POSITIVE ✅"}

for i, text in enumerate(texts[:5]):
    # Get prediction
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )
    with torch.no_grad():
        outputs = model(**inputs)

    probs = torch.softmax(outputs.logits, dim=1)[0]
    pred_label = torch.argmax(probs).item()
    confidence = probs[pred_label].item() * 100

    print(f"\nReview #{i+1}:")
    print(f"   Text       : {text[:80]}...")
    print(f"   Prediction : {label_map[pred_label]}")
    print(f"   Confidence : {confidence:.1f}%")
    print(f"   Neg/Pos    : {probs[0]:.3f} / {probs[1]:.3f}")

print("\n" + "="*60)
print("✅ ALL DONE! SHAP Analysis Complete!")
print("="*60)
```

```
✅ All libraries imported successfully!
   SHAP version     : 0.51.0
   Transformers     : 5.12.1
   Datasets         : 5.0.0
   Torch            : 2.12.1+cpu

🔄 Loading tokenizer and model...
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]
Loading weights:   0%|          | 0/104 [00:00<?, ?it/s]
```

```
`trust_remote_code` is not supported anymore.
Please check that the Hugging Face dataset 'stanfordnlp/imdb' isn't based on a lo
ading script and remove `trust_remote_code`.
If the dataset is based on a loading script, please ask the dataset author to rem
ove it and convert it to a standard format like Parquet.
✅ Model and tokenizer loaded!
✅ Prediction function defined!

🔄 Creating SHAP explainer...
✅ SHAP Explainer created!

🔄 Loading IMDB dataset...
README.md:   0%|          | 0.00/7.81k [00:00<?, ?B/s]
plain_text/train-00000-of-00001.parquet:   0%|          | 0.00/21.0M [00:00<?, ?
B/s]
plain_text/test-00000-of-00001.parquet:   0%|          | 0.00/20.5M [00:00<?, ?B/
s]
plain_text/unsupervised-00000-of-00001.p(…):   0%|          | 0.00/42.0M [00:00
<?, ?B/s]
Generating train split:   0%|          | 0/25000 [00:00<?, ? examples/s]
Generating test split:   0%|          | 0/25000 [00:00<?, ? examples/s]
Generating unsupervised split:   0%|          | 0/50000 [00:00<?, ? examples/s]
✅ Dataset loaded via 'stanfordnlp/imdb'

🔄 Preparing text samples for SHAP analysis...
✅ Prepared 10 text samples!

📝 First sample preview:
   I rented I AM CURIOUS-YELLOW from my video store because of all the controvers
y that surrounded it when it was first released in 1967. I also heard th...

🔄 Computing SHAP values for 10 samples...
   (This may take a few minutes...)

  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  10%|▉         | 1/10 [00:00<?, ?it/s]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  30%|██▉       | 3/10 [01:36<02:55, 25.10s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  40%|███▉      | 4/10 [02:21<03:20, 33.34s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  50%|████▉     | 5/10 [03:24<03:42, 44.48s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  60%|█████▉    | 6/10 [04:22<03:17, 49.40s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  70%|██████▉   | 7/10 [05:02<02:18, 46.24s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer:  80%|███████▉  | 8/10 [05:49<01:32, 46.31s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
```

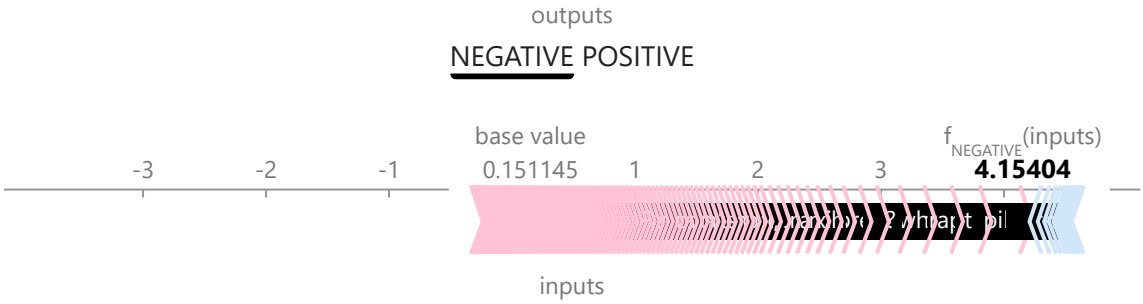
```
PartitionExplainer explainer:  90%|
|  | 9/10 [06:27<00:43, 43.70s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer: 100%|
| 10/10 [07:21<00:00, 46.92s/it]
  0%|          | 0/498 [00:00<?, ?it/s]
PartitionExplainer explainer: 11it [08:03, 48.31s/it]
✅ SHAP values computed successfully!

🔄 Generating SHAP visualizations...

📊 Plot 1: Token-level SHAP for Review #1
--------------------------------------------------
```



I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was first released in 1967. I also heard that at first it was seized by U.S. customs if it ever tried to enter this country, therefore being a fan of films considered "controversial" I really had to see this for myself.

The plot is centered around a young Swedish drama student named Lena who wants to learn everything she can about life. In particular she wants to focus her attentions to maki

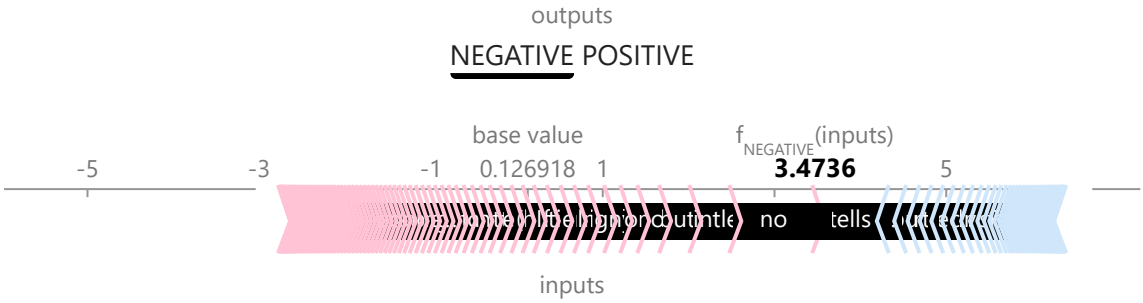
```
📊 Plot 2: Token-level SHAP for Review #2
--------------------------------------------------
```



"I Am Curious: Yellow" is a risible and pretentious steaming pile. It doesn't matter what one's political views are because this film can hardly be taken seriously on any level. As for the claim that frontal male nudity is an automatic NC-17, that isn't true. I've seen R-rated films with male nudity. Granted, they only offer some fleeting views, but where are the R-rated films with gaping vulvas and flapping labia? Nowhere, because they don't exist. The same goes for those crappy cable shows: schlongs swing

```
📊 Plot 3: Feature Importance Bar Chart
--------------------------------------------------
Bar plot error: The clustering provided by the Explanation object does not seem t
o be a partition tree, which is all shap.plots.bar supports.

📊 Plot 4: Beeswarm Summary Plot
--------------------------------------------------
Beeswarm plot skipped: The beeswarm plot does not support plotting explanations w
ith instances that have more than one dimension!

============================================================
📋 PREDICTION SUMMARY
============================================================

Review #1:
   Text       : I rented I AM CURIOUS-YELLOW from my video store because of all t
he controversy ...
   Prediction : NEGATIVE ❌
   Confidence : 99.1%
   Neg/Pos    : 0.991 / 0.009

Review #2:
   Text       : "I Am Curious: Yellow" is a risible and pretentious steaming pil
e. It doesn't ma...
   Prediction : NEGATIVE ❌
   Confidence : 99.9%
   Neg/Pos    : 0.999 / 0.001

Review #3:
   Text       : If only to avoid making this type of film in the future. This fil
m is interestin...
   Prediction : NEGATIVE ❌
   Confidence : 99.8%
   Neg/Pos    : 0.998 / 0.002

Review #4:
   Text       : This film was probably inspired by Godard's Masculin, féminin and
I urge you to ...
   Prediction : NEGATIVE ❌
   Confidence : 76.2%
   Neg/Pos    : 0.762 / 0.238

Review #5:
   Text       : Oh, brother...after hearing about this ridiculous film for umptee
n years all I c...
   Prediction : NEGATIVE ❌
   Confidence : 94.7%
   Neg/Pos    : 0.947 / 0.053

============================================================
✅ ALL DONE! SHAP Analysis Complete!
============================================================
```

In [63]:
```
# plot a sentence's explanation
shap.plots.text(shap_values[2])
```

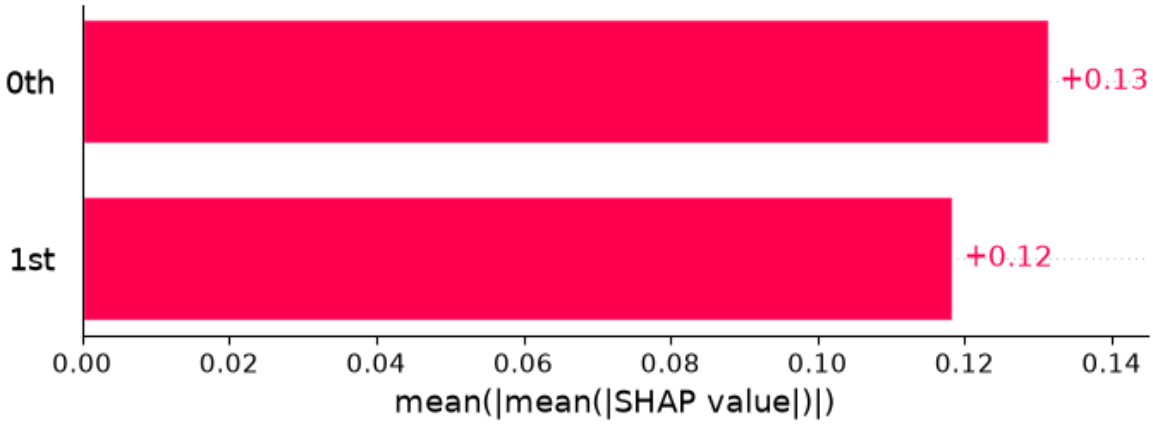


If only to avoid making this type of film in the future. This film is interesting as an experiment but tells no cogent story.

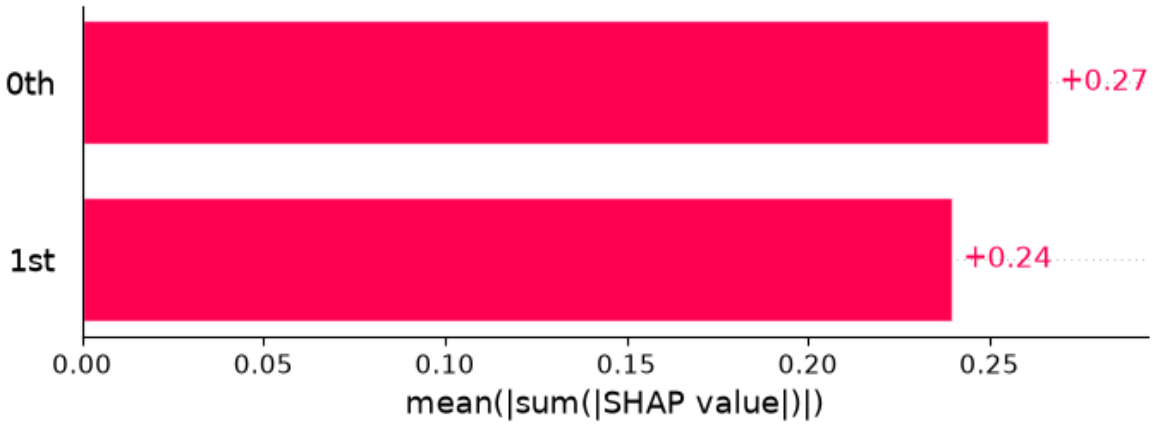
One might feel virtuous for sitting thru it because it touches on so many IMPORTANT issues but it does so without any discernable motive. The viewer comes away with no new perspectives (unless one comes up with one while one's mind wanders, as it will invariably do during this pointless film).

One might better spend one's time staring out a window at a tree grow

In [64]:
```
shap.plots.bar(shap_values.abs.mean(0))
```



In [65]:
```
shap.plots.bar(shap_values.abs.sum(0))
```



In [ ]: